

Bases de Dados Relacionais e SQL II

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercícios

Lab 0: Configuração do Ambiente

Esta aula utiliza **SQLite**, **PostgreSQL** e **Redis**. Para evitar problemas de instalação, recomendamos o uso do **Docker**.

1. Navegue até à pasta 02_support.
 2. Execute `docker compose up -d` para iniciar os serviços de PostgreSQL e Redis.
 3. Para os labs em Python, pode usar o seu ambiente local ou instalar as bibliotecas necessárias:
`pip install -r requirements.txt`
-

Lab 1: “The Need for Speed” (Impacto da Indexação)

Neste lab, irá medir o ganho de performance ao usar um índice num conjunto de dados grande.

1. **Configuração:** Execute `python3 lab_utils.py` para gerar a BD `lab_indexing.db` com 100.000 registos.
2. **Tarefa:** Abra a base de dados usando `sqlite3 lab_indexing.db`.
3. **Medição:** Ligue o temporizador e procure por um nome específico (um que não exista para forçar um scan completo).

```
.timer on  
SELECT * FROM users WHERE name = 'nao_existe';
```

4. **Otimização:** Crie um índice na coluna `name` e execute a consulta novamente.

```
CREATE INDEX idx_users_name ON users(name);  
SELECT * FROM users WHERE name = 'nao_existe';
```

5. **Observação:** Compare os tempos de execução. Quantas vezes mais rápida foi a consulta indexada?
-

Lab 2: “The Breach” (Segurança e Prepared Statements)

Neste lab, irá realizar um ataque de SQL Injection e depois corrigi-lo.

1. **Configuração:** Execute `python3 lab_utils.py` para garantir que a BD `lab_security.db` está pronta.
2. **Ataque:** Crie um pequeno script Python (ou use o existente em `solutions/`) que use concatenação de strings para uma consulta de login.
 - **Dica:** Tente fazer login como `admin` sem saber a password usando `' OR '1'='1` como `username`.

```
username = "admin' --"  
query = f"SELECT * FROM accounts WHERE username = '{username}'"
```

3. **Correção:** Recreve a função de login para usar **Prepared Statements**.
 - **Dica:** Use marcadores ? no SQLite.
`cursor.execute("SELECT * FROM accounts WHERE username = ?", (username,))`
 4. **Verificação:** Tente o ataque novamente na versão segura. Ainda funciona?
-

Lab 3: “Portable Power” (SQL Autônomo)

Compare o comportamento de bases de dados baseadas em ficheiros (SQLite) e em memória (H2/SQLite).

1. **SQLite (Ficheiro):** Crie uma tabela numa BD baseada em ficheiro, saia e reabra.
 2. **SQLite (Memória):** Execute `sqlite3 :memory:`, crie uma tabela, saia e reabra.
 3. **Observação:** O que aconteceu aos dados na base de dados apenas em memória? Quando usaria isto num projeto real?
-

Lab 4: “The Hybrid Arch” (SQL + NoSQL em Docker)

Implemente um padrão simples de “Cache-Aside” usando PostgreSQL e Redis.

1. **Configuração:** Inicie o ambiente usando `docker compose up -d`.
2. **Implementação:** Crie um script que tente procurar um utilizador no **Redis** primeiro. Se for um “Miss”, procure no **PostgreSQL** e guarde no Redis para a próxima vez.
3. **Dica (Python):**

```
import redis, psycopg2
r = redis.Redis(host='localhost', port=6379)
# Tenta Redis
data = r.get("user:1")
if not data:
    # Procura no Postgres se o Redis estiver vazio
    # ... lógica de fetch ...
    r.setex("user:1", 60, "resultado_da_db")
```

4. **Medição:** Meça o tempo para o primeiro pedido (Cache Miss) vs. o segundo pedido (Cache Hit).